

NAME: M.Greeshma

COURSE: Design and Analysis of Algorithms

CODE: CSA0619

REPORT

Q23.Minimum Number of Jumps

Given an array where each element represents the maximum number of steps that can be taken forward, determine the minimum number of jumps required to reach the last index. The

goal is to minimize the number of moves while ensuring reachability. Use Dynamic Programming or Greedy techniques to compute the optimal solution.

Input Format:

n = number of elements

arr[] = jump values

Sample Input:

n = 5

arr = 2 3 1 1 4

Expected Output:

Min Jumps = 2

Sol :

Optimal Code (Greedy, $O(n)$ time)

```
def min_jumps(arr):  
    n = len(arr)  
    if n <= 1:  
        return 0  
    if arr[0] == 0:  
        return float('inf') # cannot move
```

```
maxReach = arr[0]
```

```
steps = arr[0]
```

```
jumps = 1
```

```
for i in range(1, n):
```

```
    if i == n - 1:
```

```
        return jumps
```

```
    maxReach = max(maxReach, i + arr[i])
```

```
    steps -= 1
```

```
if steps == 0:
```

```
    jumps += 1
```

```
    if i >= maxReach:
```

```
        return float('inf') # unreachable
```

```
    steps = maxReach - i
```

```
return float('inf')
```

```
main.py
1 def min_jumps(arr):
2     n = len(arr)
3
4     # If array has only one element, no jumps needed
5     if n <= 1:
6         return 0
7
8     # If first element is 0, we can't move anywhere
9     if arr[0] == 0:
10        return -1 # unreachable
11
12    # Initialize variables
13    maxReach = arr[0] # farthest we can reach
14    steps = arr[0]    # steps we can still take
15    jumps = 1        # we start with one jump
16
17    for i in range(1, n):
18        # If we've reached the end
19        if i == n - 1:
20            return jumps
21
22        # Update maxReach
23        maxReach = max(maxReach, i + arr[i])
24
25        # Use a step
26        steps -= 1
27
28        # If no steps left, we must jump
29        if steps == 0:
30            jumps += 1
31        # If current index is beyond maxReach, can't move further
```

Min Jumps = 2

...Program finished with exit code 0
Press ENTER to exit console

1. Problem Definition

- You are given an array where each element represents the maximum steps you can move forward from that position.
- The task is to find the **minimum number of jumps** needed to reach the last index.

2. Key Observations

- Each element gives a "range of reachability."
- The challenge is to minimize jumps while ensuring you don't get stuck.

3. Approaches

- **Dynamic Programming (DP):**
 - Try all possible jumps from each index.
 - Store the minimum jumps needed to reach each position.
 - Time complexity: **$O(n^2)$** (because for each index you may check all reachable indices).
- **Greedy Approach (Optimal):**
 - Track the **farthest index** you can reach (maxReach).
 - Track how many steps you can still take before making another jump (steps).
 - When steps becomes 0, you must make a jump and reset steps to cover the next range.
 - Time complexity: **$O(n)$** .

4. Variables in Greedy Solution

- **maxReach:** The farthest index reachable at any point.
- **steps:** Remaining steps before you must jump.
- **jumps:** Number of jumps taken so far.

5. Algorithm Flow

- Start with jumps = 1 (since you must jump from the first element).
- Iterate through the array:
 - Update maxReach with the farthest index reachable.
 - Decrease steps by 1.
 - If steps == 0, increment jumps and reset steps = maxReach - currentIndex.
- If you reach the last index, return jumps.

6. Edge Cases

- If the array has only one element → 0 jumps.

- If the first element is 0 → unreachable (-1).
- If at any point $i \geq \text{maxReach}$ → unreachable.

COMPARISON

Approach	Time Complexity	Space Complexity	Suitability
Dynamic Programming	$O(n^2)$	$O(n)$	Educational, but slow
Greedy	$O(n)$	$O(1)$	Best choice for large arrays